

Universidad Autónoma de Sinaloa
Facultad de Informática
Culiacán



Gestión de Seguridad y Desempeño de Bases de Datos

Alumno: Zamora Serrano Irvin Donovan

Profesor: Fidel Bojórquez Solis

Grado: 4 **Grupo:** 3

Práctica 1: Protección y Privacidad de Datos

Fecha de entrega: 23 de octubre de 2026

Actividad 3 - Seudonimizacion

Objetivo

Separar los datos identificables (PII) de los datos que se usan para análisis, sustituyendo el identificador real de cada cliente por un token aleatorio (UUID). La tabla que relaciona el token con el cliente real queda en un esquema restringido, y el rol `analista` solo puede consultar los datos seudonimizados.

Paso 1. Creación de los esquemas

Se crean dos esquemas: `pii`, que guardará la tabla de correspondencia entre token y cliente, y `analitica`, que contendrá los datos seudonimizados.

```
CREATE SCHEMA pii;
CREATE SCHEMA analitica;
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=# CREATE SCHEMA pii;
CREATE SCHEMA
postgres=# CREATE SCHEMA analitica;
CREATE SCHEMA
postgres=#
```

Paso 2. Creación de la tabla de mapeo

Se crea la tabla `pii.mapa_cliente`, que relaciona cada token (UUID) con el `id` real del cliente en `public.clientes`.

```
CREATE TABLE pii.mapa_cliente (
    token UUID PRIMARY KEY,
    cliente_id BIGINT UNIQUE REFERENCES public.clientes(id)
);
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=# CREATE TABLE pii.mapa_cliente (
    token UUID PRIMARY KEY,
    cliente_id BIGINT UNIQUE REFERENCES public.clientes(id)
);
CREATE TABLE
postgres=#
```

Paso 3. Generación de los tokens

Se habilita la extensión `pgcrypto` y se insertan en la tabla de mapeo los tokens aleatorios generados con `gen_random_uuid()`, uno por cada cliente.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

INSERT INTO pii.mapa_cliente (token, cliente_id)
SELECT gen_random_uuid(), id FROM clientes;

SELECT * FROM pii.mapa_cliente;
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=#
postgres=# CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION
postgres=# INSERT INTO pii.mapa_cliente (token, cliente_id)
postgres-# SELECT gen_random_uuid(), id FROM clientes;
INSERT 0 5
postgres=# SELECT * FROM pii.mapa_cliente;
                token                 | cliente_id
--------------------------------------+------------
 28f12eba-f669-4e67-9a78-859e1d71e806 |          1
 70d27792-bca8-452e-87b4-f17a04edc577 |          2
 b4cf2078-d86c-40da-b99a-ffb1a0e5eb86 |          3
 fd6b30f8-96f2-42b6-854e-d4c449171c66 |          4
 b30f1357-74e7-4a52-8ec5-7bdb29128a69 |          5
(5 rows)

postgres=#
```

Paso 4. Creación de la tabla analítica

Se crea la tabla `analitica.clientes`, que reemplaza el identificador real por el token y conserva únicamente los campos no identificadores (`edad`, `ciudad`, `codigo_postal` y `fecha_alta`). Se omiten `nombre`, `correo`, `telefono` y `rfc`.

```
CREATE TABLE analitica.clientes AS
SELECT
    m.token AS cliente_token,
    c.edad,
    c.ciudad,
    c.codigo_postal,
    c.fecha_alta
FROM pii.mapa_cliente m
JOIN clientes c ON c.id = m.cliente_id;
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=# CREATE TABLE analitica.clientes AS
SELECT
    m.token AS cliente_token,
    c.edad,
    c.ciudad,
    c.codigo_postal,
    c.fecha_alta
FROM pii.mapa_cliente m
JOIN clientes c ON c.id = m.cliente_id;
SELECT 5
postgres=#
```

Verificación del contenido de la tabla recién creada:

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=# SELECT * FROM analitica.clientes;
            cliente_token             | edad |   ciudad   | codigo_postal | fecha_alta
--------------------------------------+------+------------+---------------+------------
 28f12eba-f669-4e67-9a78-859e1d71e806 |   41 | Culiacán   | 80000         | 2026-10-24
 70d27792-bca8-452e-87b4-f17a04edc577 |   35 | Culiacán   | 80100         | 2026-10-24
 b4cf2078-d86c-40da-b99a-ffb1a0e5eb86 |   47 | Mazatlán   | 82000         | 2026-10-24
 fd6b30f8-96f2-42b6-854e-d4c449171c66 |   34 | Los Mochis | 81200         | 2026-10-24
 b30f1357-74e7-4a52-8ec5-7bdb29128a69 |   38 | Culiacán   | 80200         | 2026-10-24
(5 rows)

postgres=#
```

Paso 5. Comparación de los tres conjuntos de datos

Se consultan las tres tablas para comprobar la separación de los datos.

Datos originales (`public.clientes`): contienen los datos personales completos.

```
SELECT * FROM public.clientes;
```

```
postgres=# SELECT * FROM public.clientes;
 id |    nombre     |          correo           |  telefono  |      rfc      | edad |   ciudad   | codigo_postal | fecha_alta
----+---------------+---------------------------+------------+---------------+------+------------+---------------+------------
  1 | Ana López     | ana.lopez@example.test    | 6671234567 | LOAA850315ABC |   41 | Culiacán   | 80000         | 2026-10-24
  2 | Carlos Pérez  | carlos.perez@example.test | 6672345678 | PECA900720DEF |   35 | Culiacán   | 80100         | 2026-10-24
  3 | María García  | maria.garcia@example.test | 6673456789 | GAMA780912GHI |   47 | Mazatlán   | 82000         | 2026-10-24
  4 | Luis Torres   | luis.torres@example.test  | 6674567890 | TOLU920201JKL |   34 | Los Mochis | 81200         | 2026-10-24
  5 | Laura Sánchez | laura.sanchez@example.test| 6675678901 | SALA880525MNO |   38 | Culiacán   | 80200         | 2026-10-24
(5 rows)

postgres=#
```

Tabla de mapeo (`pii.mapa_cliente`): relaciona cada token con el cliente real y es la única forma de revertir la seudonimización.

```
SELECT * FROM pii.mapa_cliente;
```

```
postgres=# SELECT * FROM pii.mapa_cliente;
                token                 | cliente_id
--------------------------------------+------------
 28f12eba-f669-4e67-9a78-859e1d71e806 |          1
 70d27792-bca8-452e-87b4-f17a04edc577 |          2
 b4cf2078-d86c-40da-b99a-ffb1a0e5eb86 |          3
 fd6b30f8-96f2-42b6-854e-d4c449171c66 |          4
 b30f1357-74e7-4a52-8ec5-7bdb29128a69 |          5
(5 rows)

postgres=#
```

Datos seudonimizados (`analitica.clientes`): solo muestran el token y los datos no identificadores.

```
SELECT * FROM analitica.clientes;
```

```
postgres=#
postgres=# SELECT * FROM analitica.clientes;
           cliente_token             | edad |   ciudad   | codigo_postal | fecha_alta
-------------------------------------+------+------------+---------------+------------
 28f12eba-f669-4e67-9a78-859e1d71e806 |   41 | Culiacán   | 80000         | 2026-10-24
 70d27792-bca8-452e-87b4-f17a04edc577 |   35 | Culiacán   | 80100         | 2026-10-24
 b4cf2078-d86c-40da-b99a-ffb1a0e5eb86 |   47 | Mazatlán   | 82000         | 2026-10-24
 fd6b30f8-96f2-42b6-854e-d4c449171c66 |   34 | Los Mochis | 81200         | 2026-10-24
 b30f1357-74e7-4a52-8ec5-7bdb29128a69 |   38 | Culiacán   | 80200         | 2026-10-24
(5 rows)
```

Paso 6. Creación del rol `analista` y asignación de permisos

Se crea el rol `analista` (sin inicio de sesión) y se le otorga acceso únicamente al esquema `analitica` y lectura de la tabla `analitica.clientes`.

```
CREATE ROLE analista NOLOGIN;
GRANT USAGE ON SCHEMA analitica TO analista;
GRANT SELECT ON analitica.clientes TO analista;
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=# CREATE ROLE analista NOLOGIN;
CREATE ROLE
postgres=# GRANT USAGE ON SCHEMA analitica TO analista;
GRANT
postgres=# GRANT SELECT ON analitica.clientes TO analista;
GRANT
postgres=#
```

Paso 7. Verificación del control de acceso

Se asume el rol `analista` con `SET ROLE` y se comprueba que:

- **Sí puede** consultar `analitica.clientes`.
- **No puede** consultar `pii.mapa_cliente` (error: *permission denied for schema pii*).
- **No puede** consultar `public.clientes` (error: *permission denied for table clientes*).

```
SET ROLE analista;
SELECT * FROM analitica.clientes;
SELECT * FROM pii.mapa_cliente;
SELECT * FROM public.clientes;
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=#
postgres=# SET ROLE analista;
SET
postgres=> SELECT * FROM analitica.clientes;
            cliente_token             | edad |   ciudad   | codigo_postal | fecha_alta
--------------------------------------+------+------------+---------------+------------
 28f12eba-f669-4e67-9a78-859e1d71e806 |   41 | Culiacán   | 80000         | 2026-10-24
 70d27792-bca8-452e-87b4-f17a04edc577 |   35 | Culiacán   | 80100         | 2026-10-24
 b4cf2078-d86c-40da-b99a-ffb1a0e5eb86 |   47 | Mazatlán   | 82000         | 2026-10-24
 fd6b30f8-96f2-42b6-854e-d4c449171c66 |   34 | Los Mochis | 81200         | 2026-10-24
 b30f1357-74e7-4a52-8ec5-7bdb29128a69 |   38 | Culiacán   | 80200         | 2026-10-24
(5 rows)

postgres=> SELECT * FROM pii.mapa_cliente;
ERROR:  permission denied for schema pii
LINE 1: SELECT * FROM pii.mapa_cliente;
                      ^
postgres=> SELECT * FROM public.clientes;
ERROR:  permission denied for table clientes
postgres=>
```

Conclusión

La seudonimización permite que el rol `analista` trabaje con datos útiles para el análisis (edad, ciudad, código postal y fecha de alta) sin poder identificar a ningún cliente. La tabla de mapeo en el esquema `pii` queda aislada, por lo que solo los usuarios autorizados pueden reidentificar a las personas.